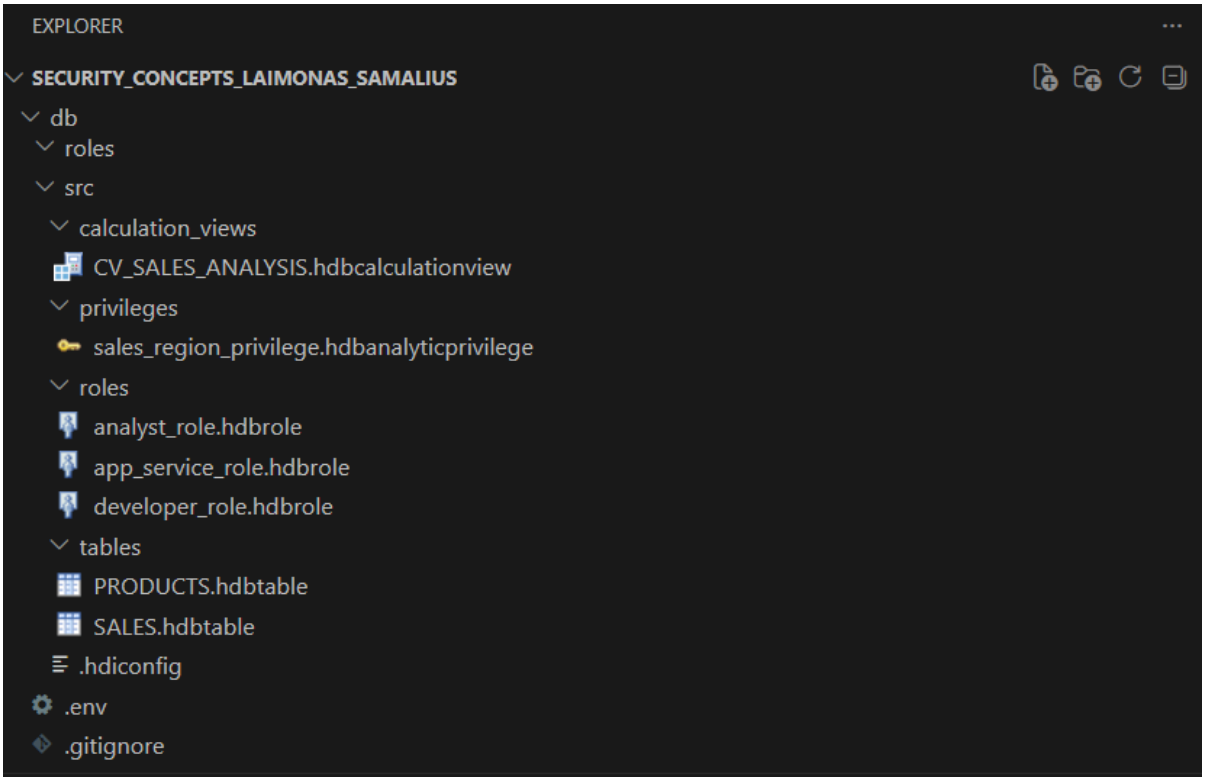
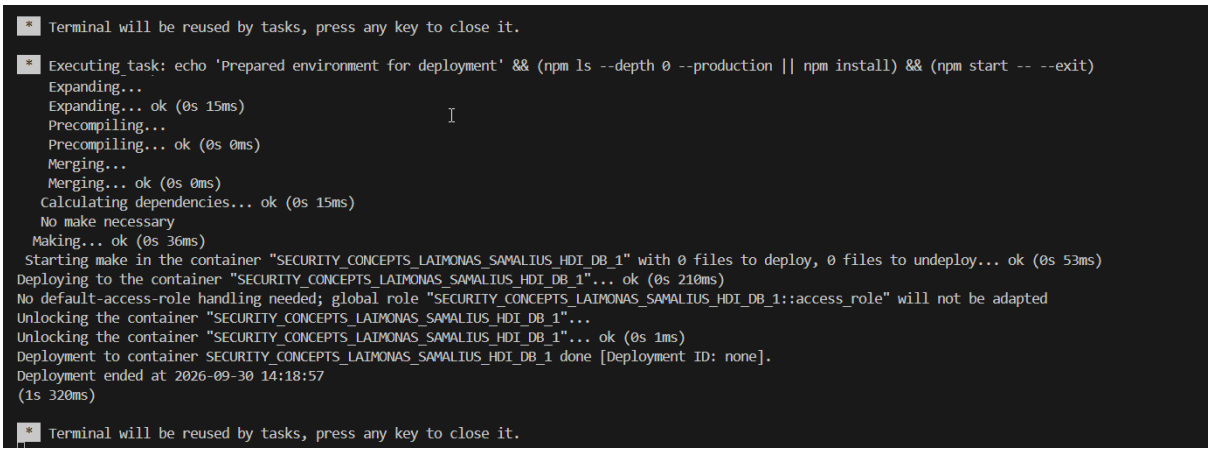
Security Concepts

Phase 1: Define HDI Roles and Security Artifacts

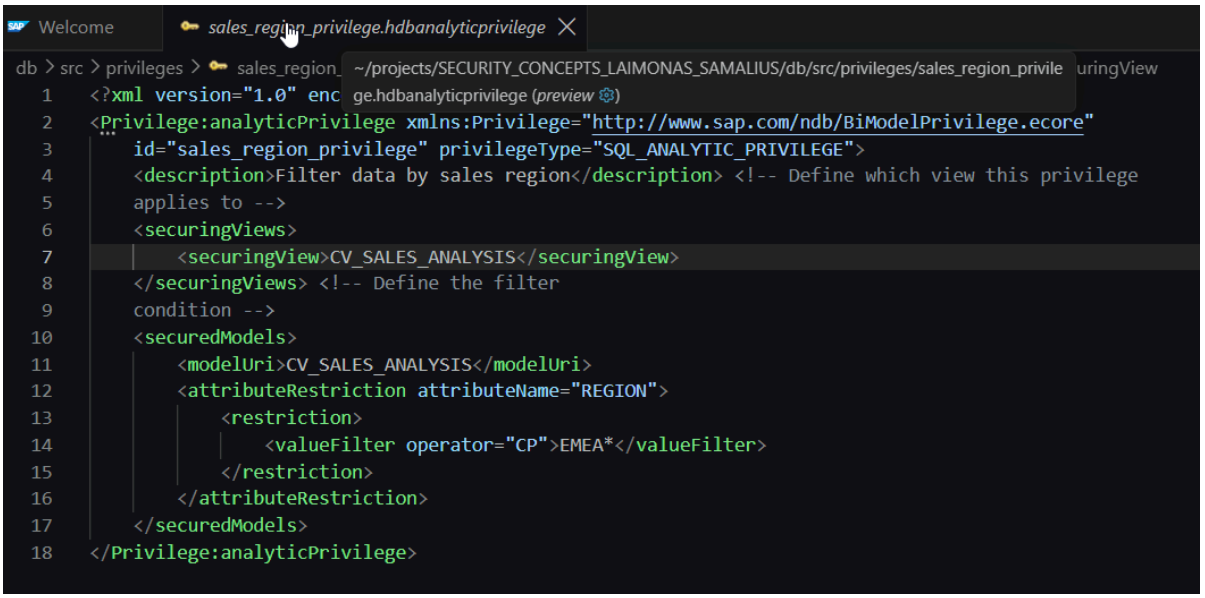
Project structure screenshot



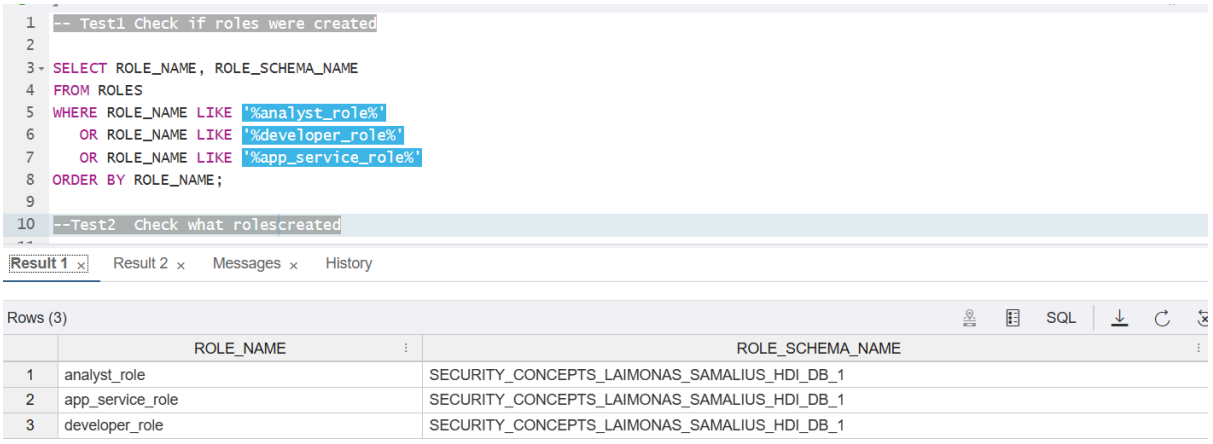
Deployment success screenshot



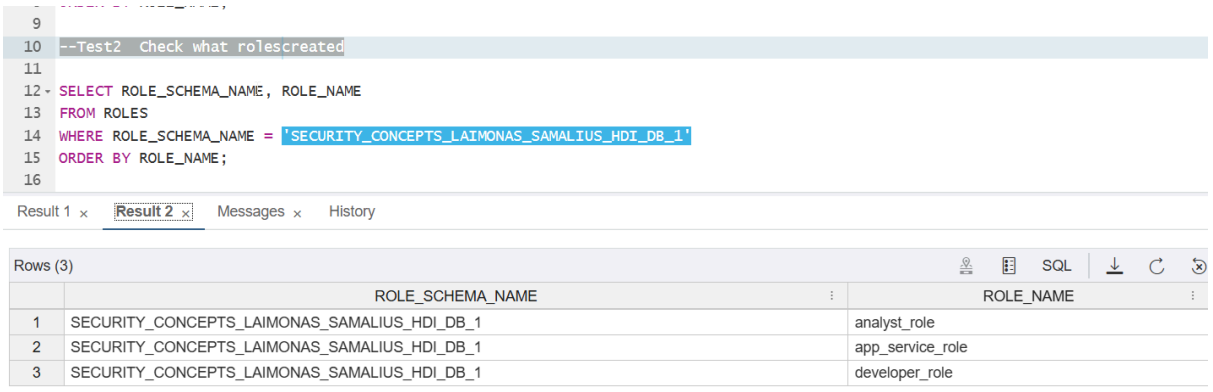
Screenshot of .hdbanalyticprivilege file



Screenshots of verification queries showing created rolesTest1



Test2



Explanation of each role's purpose and privileges:

Role	Purpose	Data access	Row filter	Typical user
`analyst_role`	Read-only reporting	SELECT	Yes (`sales_region_privilege`)	Business analysts, report viewers
`developer_role`	Development and testing	SELECT, INSERT, UPDATE, DELETE, EXECUTE	No	Developers, data engineers
`app_service_role`	Technical access for applications	SELECT	No	Backend services, APIs, automated reports

analyst_role: read-only with row-level security

Purpose: access to view data without being able to change it, and only for the rows they are allowed to see.

Privileges:

- `SELECT` on the container schema lets the user query tables and views.
- `sales_region_privilege` is an analytic privilege that filters rows. With the `EMEA` filter, an analyst querying `CV_SALES_ANALYSIS` sees only EMEA rows. The filtering is silent: there is no error, the other rows just aren't returned.
- It has no INSERT, UPDATE, DELETE or DDL rights, so any attempt to change data fails with "insufficient privilege".

Why: reporting users need to read data, and applying the filter through the role means it can't be forgotten or bypassed by writing a different query.

developer_role: full data manipulation

Purpose: let developers build and test against real data structures.

Privileges:

- `SELECT`, `INSERT`, `UPDATE`, `DELETE` allow full data manipulation on the container's objects.
- `EXECUTE` allows running stored procedures and functions.
- It has no analytic privilege, so developers see all rows. This is deliberate, because developers need unfiltered data to test properly.
- It has no DDL rights. In HDI, tables and views are created only by deploying design-time files, not by a user running `CREATE TABLE`, so developers change structure through version-controlled files.

Why: developers get the access they need to work, while structure changes stay controlled through deployment. It should be used in development and test systems, not given to end users in production.

app_service_role: minimal technical access

Purpose: give a backend service or API the smallest access it needs to read data.

Privileges:

- SELECT only, so no changes are possible.
- The object privilege on the calculation view CV_SALES_ANALYSIS names the exact object the service may read.

Phase 2: Peer-to-Peer Access Exchange

My Information:

Laimonas Samalius

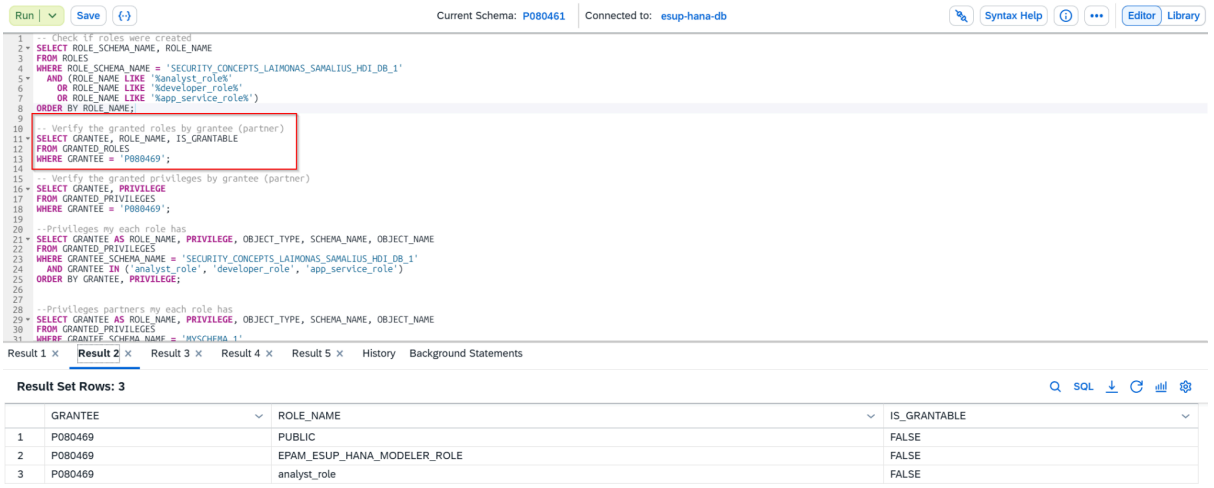
Database Username: **P080461**
Schema Name: **SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1**
Calculation View: **CV_SALES_ANALYSIS**

Task partner information:

Sandis Sarkovskis

Database Username:
HDI Schema Name: **MYSCHEMA_1**
Calculation View: **CV_SALES_SECURED**

- Exchanged usernames and schema names with partner - YES
- Granted analyst_role to partner's user - YES was granted by Timur Pitsko
- Received confirmation from partner that they granted you access - YES was confirmed by Timur Pitsko
- GRANT statements executed - YES by Timur Pitsko
- Verification query results screenshots proving successful grants:



```
-- Check if roles were created
SELECT ROLE_SCHEMA_NAME, ROLE_NAME
FROM ROLES
WHERE ROLE_SCHEMA_NAME = 'SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1'
  AND (ROLE_NAME LIKE '%analyst_role%'
    OR ROLE_NAME LIKE '%developer_role%'
    OR ROLE_NAME LIKE '%app_service_role%')
ORDER BY ROLE_NAME;

-- Verify the granted roles by grantee (partner)
SELECT GRANTEE, ROLE_NAME, IS_GRANTABLE
FROM GRANTED_ROLES
WHERE GRANTEE = 'P080469';

-- Verify the granted privileges by grantee (partner)
SELECT GRANTEE, PRIVILEGE
FROM GRANTED_PRIVILEGES
WHERE GRANTEE = 'P080469';

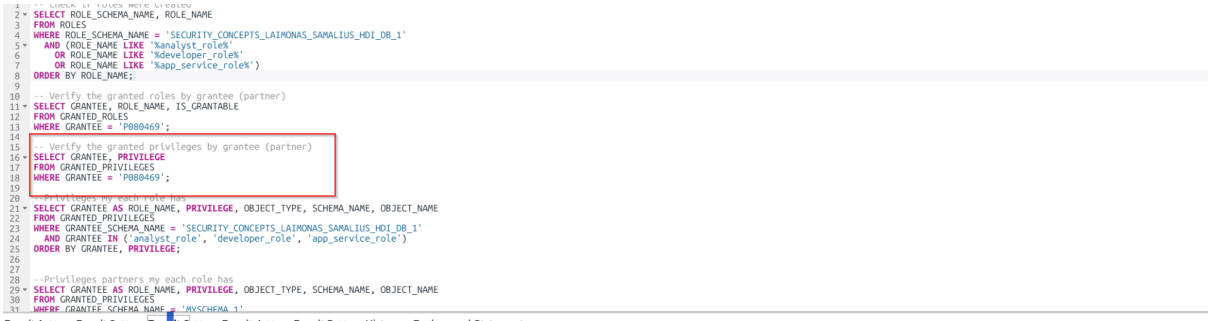
--Privileges my each role has
SELECT GRANTEE AS ROLE_NAME, PRIVILEGE, OBJECT_TYPE, SCHEMA_NAME, OBJECT_NAME
FROM GRANTED_PRIVILEGES
WHERE GRANTEE_SCHEMA_NAME = 'SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1'
  AND GRANTEE IN ('analyst_role', 'developer_role', 'app_service_role')
ORDER BY GRANTEE, PRIVILEGE;

--Privileges partners my each role has
SELECT GRANTEE AS ROLE_NAME, PRIVILEGE, OBJECT_TYPE, SCHEMA_NAME, OBJECT_NAME
FROM GRANTED_PRIVILEGES
WHERE GRANTEE_SCHEMA_NAME = 'MYSCHEMA_1'
```

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | History | Background Statements

Result Set Rows: 3

	GRANTEE	ROLE_NAME	IS_GRANTABLE
1	P080469	PUBLIC	FALSE
2	P080469	EPAM_ESUP_HANA_MODELER_ROLE	FALSE
3	P080469	analyst_role	FALSE

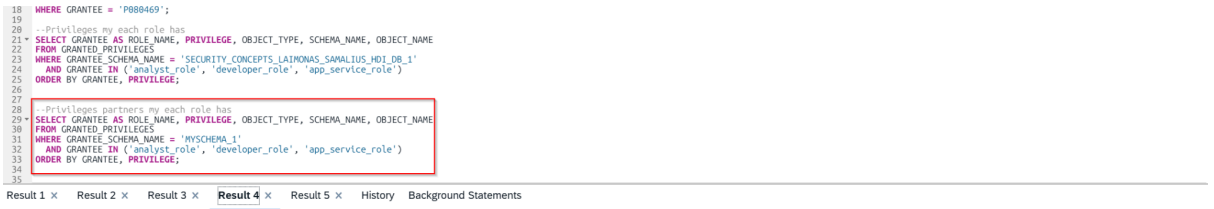


```
-- Verify the granted privileges by grantee (partner)
SELECT GRANTEE, PRIVILEGE
FROM GRANTED_PRIVILEGES
WHERE GRANTEE = 'P080469';
```

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | History | Background Statements

Result Set Rows: 1

	GRANTEE	PRIVILEGE
1	P080469	CREATE ANY



```
WHERE GRANTEE = 'P080469';

--Privileges my each role has
SELECT GRANTEE AS ROLE_NAME, PRIVILEGE, OBJECT_TYPE, SCHEMA_NAME, OBJECT_NAME
FROM GRANTED_PRIVILEGES
WHERE GRANTEE_SCHEMA_NAME = 'SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1'
  AND GRANTEE IN ('analyst_role', 'developer_role', 'app_service_role')
ORDER BY GRANTEE, PRIVILEGE;

--Privileges partners my each role has
SELECT GRANTEE AS ROLE_NAME, PRIVILEGE, OBJECT_TYPE, SCHEMA_NAME, OBJECT_NAME
FROM GRANTED_PRIVILEGES
WHERE GRANTEE_SCHEMA_NAME = 'MYSCHEMA_1'
  AND GRANTEE IN ('analyst_role', 'developer_role', 'app_service_role')
ORDER BY GRANTEE, PRIVILEGE;
```

Result 1 | Result 2 | Result 3 | Result 4 | Result 5 | History | Background Statements

Result Set Rows: 9

	ROLE_NAME	PRIVILEGE	OBJECT_TYPE	SCHEMA_NAME	OBJECT_NAME
1	analyst_role	EXECUTE	SQLANALYTICALPRIVILEGE	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	sales_region_privilege
2	analyst_role	SELECT	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
3	app_service_role	SELECT	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
4	app_service_role	SELECT	VIEW	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	CV_SALES_ANALYSIS
5	developer_role	DELETE	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
6	developer_role	EXECUTE	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
7	developer_role	INSERT	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
8	developer_role	SELECT	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL
9	developer_role	UPDATE	SCHEMA	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1	NULL

```
--Privileges partners my each role has
SELECT GRANTEE AS ROLE_NAME, PRIVILEGE, OBJECT_TYPE, SCHEMA_NAME, OBJECT_NAME
FROM GRANTED_PRIVILEGES
WHERE GRANTEE_SCHEMA_NAME = 'MYSCHEMA_1'
  AND GRANTEE IN ('analyst_role', 'developer_role', 'app_service_role')
ORDER BY GRANTEE, PRIVILEGE;
```

Result 1 × Result 2 × Result 3 × Result 4 × **Result 5** × History Background Statements

Result Set Rows: 8

	ROLE_NAME	PRIVILEGE	OBJECT_TYPE	SCHEMA_NAME	OBJECT_NAME
1	analyst_role	EXECUTE	SQLANALYTICALPRIVILEGE	MYSCHEMA_1	sales_region_privilege
2	analyst_role	SELECT	SCHEMA	MYSCHEMA_1	NULL
3	app_service_role	SELECT	VIEW	MYSCHEMA_1	CV_SALES_WITH_ANALYTICS
4	developer_role	DELETE	SCHEMA	MYSCHEMA_1	NULL
5	developer_role	EXECUTE	SCHEMA	MYSCHEMA_1	NULL
6	developer_role	INSERT	SCHEMA	MYSCHEMA_1	NULL
7	developer_role	SELECT	SCHEMA	MYSCHEMA_1	NULL
8	developer_role	UPDATE	SCHEMA	MYSCHEMA_1	NULL

Phase 3: Verification and Audit

Step 3.1: Test Read Access:

```
-- Part3
-- Step 3.1:
-- Test to Query a calculation view
SELECT * FROM "MYSCHEMA_1"."CV_SALES_SECURED"
LIMIT 10;

-- Test 2: Count records
SELECT COUNT(*) AS TOTAL_RECORDS
FROM "MYSCHEMA_1"."CV_SALES_SECURED";
```

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × **Result 6** × Result 7 × History Background Statements

Result Set Rows: 10

	CUSTOMER_NAME	REGION	CUSTOMER_ID	SALE_ID	PRODUCT_ID	SALE_DATE	PRICE	COST	CATEGORY	PRODUCT_NAME	YEAR	SALE_SIZE	TOTAL_SALES	EMEA_SALES	APAC
1	Carlos Garcia	Europe	3	3	3	2023-03-10	29.99	15.00	Electronics	Wireless Mouse	2023	Standard	29.99	29.99	NULL
2	Maria Rossi	Europe	7	15	2	2023-06-15	149.99	80.00	Furniture	Desk Chair	2023	Standard	149.99	149.99	NULL
3	Emma Johnson	Europe	2	18	8	2023-10-12	12.99	5.00	Stationery	Pen Set	2023	Standard	12.99	12.99	NULL
4	Sophie Mueller	Europe	5	21	1	2024-01-10	999.99	600.00	Electronics	Laptop Pro	2024	High	999.99	999.99	NULL
5	Carlos Garcia	Europe	3	27	5	2024-03-15	5.00	2.00	Stationery	Notebook	2024	Standard	4.99	4.99	NULL
6	Maria Rossi	Europe	7	7	7	2023-09-08	149.99	90.00	Furniture	Bookshelf	2023	Standard	149.99	149.99	NULL
7	Emma Johnson	Europe	2	10	9	2023-12-20	89.99	50.00	Electronics	Webcam HD	2023	Standard	89.99	89.99	NULL
8	Sophie Mueller	Europe	5	13	3	2023-03-10	29.99	15.00	Electronics	Wireless Mouse	2023	Standard	29.99	29.99	NULL
9	Carlos Garcia	Europe	3	19	9	2023-11-25	89.99	50.00	Electronics	Webcam HD	2023	Standard	89.99	89.99	NULL
10	Emma Johnson	Europe	2	2	2	2023-02-20	149.99	80.00	Furniture	Desk Chair	2023	Standard	149.99	149.99	NULL

```
-- Test 2: Count records
SELECT COUNT(*) AS TOTAL_RECORDS
FROM "MYSCHEMA_1"."CV_SALES_SECURED";
```

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × Result 6 × **Result 7** × History Background Statements

Result Set Rows: 1

	TOTAL_RECORDS
1	15

Step 3.2: Verify Row-Level Security (Analytic Privilege)

```
--Step 3.2: Verify Row-Level Security (Analytic Privilege)
-- Query the secured view and check what data you can see
SELECT REGION, COUNT(*) AS ROW_COUNT, SUM(TOTAL_SALES) AS TOTAL_AMOUNT
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
GROUP BY REGION
ORDER BY REGION;

-- If privilege filters for EMEA region, you should only see EMEA data
-- Try to see all regions (this should still only show filtered data)
SELECT DISTINCT REGION
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
ORDER BY REGION;

-- Try to see all regions (this should still only show filtered data)
SELECT *
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
```

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × Result 6 × Result 7 × **Result 8** × Result 9 × Result 10 × History Background Statements

Result Set Rows: 1

	REGION	ROW_COUNT	TOTAL_AMOUNT
1	Europe	15	3431.85

```
-- If privilege filters for EMEA region, you should only see EMEA data
-- Try to see all regions (this should still only show filtered data)
SELECT DISTINCT REGION
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
ORDER BY REGION;

SELECT *
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
```

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × Result 6 × Result 7 × Result 8 × **Result 9** × Result 10 × History Background Statements

Result Set Rows: 1

SQL

	REGION
1	Europe

```
ORDER BY REGION;

SELECT *
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
```

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × Result 6 × Result 7 × Result 8 × Result 9 × **Result 10** × History Background Statements

Result Set Rows: 15

SQL

	CUSTOMER_NAME	REGION	CUSTOMER_ID	SALE_ID	PRODUCT_ID	SALE_DATE	PRICE	COST	CATEGORY	PRODUCT_NAME	YEAR	SALE_SIZE	TOTAL_SALES	EMEA_SALES	APA
1	Carlos Garcia	Europe	3	3	3	2023-03-10	29.99	15.00	Electronics	Wireless Mouse	2023	Standard	29.99	29.99	NUI
2	Maria Rossi	Europe	7	15	2	2023-06-15	149.99	80.00	Furniture	Desk Chair	2023	Standard	149.99	149.99	NUI
3	Emma Johnson	Europe	2	18	8	2023-10-12	12.99	5.00	Stationery	Pen Set	2023	Standard	12.99	12.99	NUI
4	Sophie Mueller	Europe	5	21	1	2024-01-10	999.99	600.00	Electronics	Laptop Pro	2024	High	999.99	999.99	NUI
5	Carlos Garcia	Europe	3	27	5	2024-03-15	5.00	2.00	Stationery	Notebook	2024	Standard	4.99	4.99	NUI
6	Maria Rossi	Europe	7	7	7	2023-09-08	149.99	90.00	Furniture	Bookshelf	2023	Standard	149.99	149.99	NUI
7	Emma Johnson	Europe	2	10	9	2023-12-20	89.99	50.00	Electronics	Webcam HD	2023	Standard	89.99	89.99	NUI
8	Sophie Mueller	Europe	5	13	3	2023-03-10	29.99	15.00	Electronics	Wireless Mouse	2023	Standard	29.99	29.99	NUI
9	Carlos Garcia	Europe	3	19	9	2023-11-25	89.99	50.00	Electronics	Webcam HD	2023	Standard	89.99	89.99	NUI
10	Emma Johnson	Europe	2	2	2	2023-02-20	149.99	80.00	Furniture	Desk Chair	2023	Standard	149.99	149.99	NUI
11	Sophie Mueller	Europe	5	5	1	2023-06-15	999.99	600.00	Electronics	Laptop Pro	2023	High	999.99	999.99	NUI
12	Carlos Garcia	Europe	3	11	10	2023-01-15	3.99	1.50	Stationery	Sticky Notes	2023	Standard	3.99	3.99	NUI
13	Maria Rossi	Europe	7	23	3	2024-05-20	29.99	15.00	Electronics	Wireless Mouse	2024	Standard	29.99	29.99	NUI
14	Emma Johnson	Europe	2	26	4	2024-01-10	599.99	350.00	Furniture	Standing Desk	2024	High	599.99	599.99	NUI

Step 3.3: Test Restricted Operations (Negative Testing)

```
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
GROUP BY REGION
ORDER BY REGION;

-- If privilege filters for EMEA region, you should only see EMEA data
-- Try to see all regions (this should still only show filtered data)
SELECT DISTINCT REGION
FROM "MYSCHEMA_1"."CV_SALES_SECURED"
ORDER BY REGION;


-- Step 3.3: Test Restricted Operations (Negative Testing)
-- This should FAIL with "insufficient privilege" error
-- Expected error: insufficient privilege
INSERT INTO "MYSCHEMA_1".PRODUCTS VALUES (11, 'Desk Lamp', 'Furniture', 39.99, 75

-- This should FAIL with "insufficient privilege" error
-- Expected error: insufficient privilege
UPDATE "MYSCHEMA_1".PRODUCTS
SET "PRICE" = 100.99
WHERE "ID" = 1;

-- This should FAIL with "insufficient privilege" error
-- Expected error: insufficient privilege
UPDATE "MYSCHEMA_1"."CV_SALES_SECURED"
SET PRICE = 100.99
WHERE "PRODUCT_ID" = 1;
```

SQL Execution Error

(dberror) [258]: insufficient privilege: Detailed info for this error can be found with guid 'AD4DBCC3B082B9409C28D86862AC4ABD'

Copy Skip Skip All Stop Execution

```
-- This should FAIL with "insufficient privilege" error
-- Expected error: insufficient privilege
UPDATE "MYSCHEMA_1".PRODUCTS
SET "PRICE" = 100.99
WHERE "ID" = 1;

-- This should FAIL with "insufficient privilege" error
-- Expected error: insufficient privilege
UPDATE "MYSCHEMA_1"."CV_SALES_SECURED"
SET PRICE = 100.99
WHERE "PRODUCT_ID" = 1;
```

SQL Execution Error

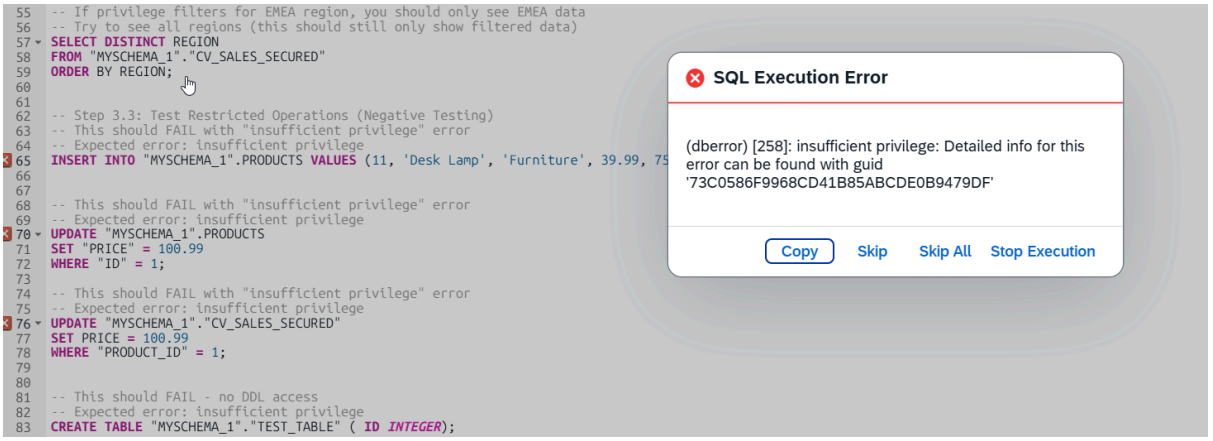
(dberror) [258]: insufficient privilege: Detailed info for this error can be found with guid 'E576E0B05EC60243990F6E272334942E'

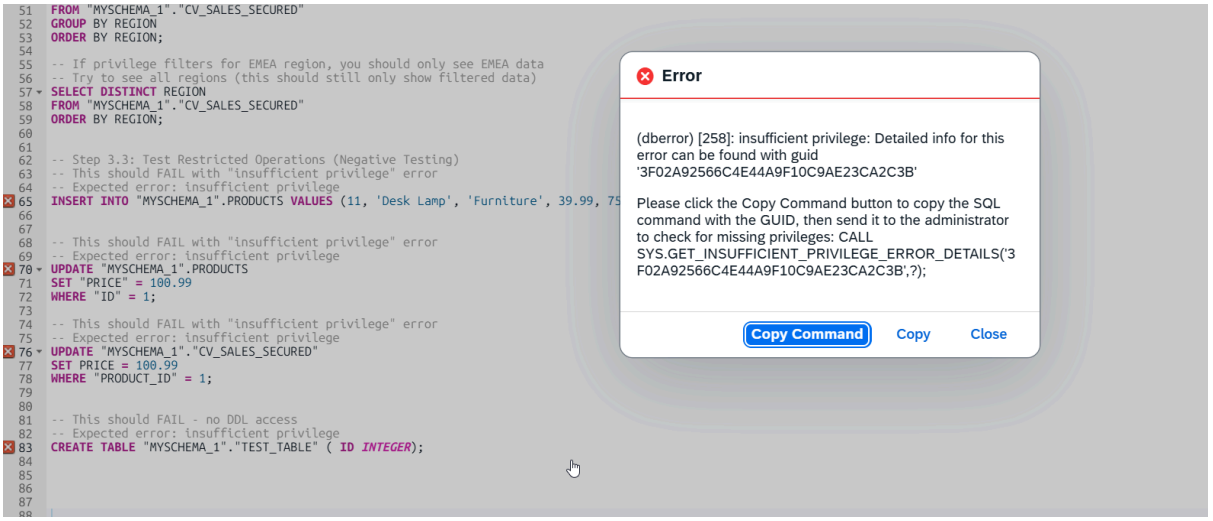
Copy Skip Skip All Stop Execution

Result 1 × Result 2 × Result 3 × Result 4 × Result 5 × Result 6 ×

Result Set Rows: 3

	ROLE_SCHEMA_NAME
1	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1
2	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1
3	SECURITY_CONCEPTS_LAIMONAS_SAMALIUS_HDI_DB_1





Step 3.4: Document Security Audit Results

Read Access
View schemaobjects
Success
✓ Can seetables/views
✅PASS

SELECT query
Success
✓ Data returned
✅PASS

COUNT records
Success
✓ Count returned
✅PASS

Row-LevelSecurity
Query filtered view
Only Europe data visible

✓ Only Europe returned
✅PASS

Try to see other regions
Automatically filtered out
✓ Other regions invisible
✅PASS

RestrictedOperations
INSERT
Denied - insufficient privilege
✗ Error 258
✅PASS

UPDATE
Denied - insufficient privilege
✗ Error 258
✅PASS

DELETE
Denied - insufficient privilege
✗ Error 258
✅PASS

CREATE TABLE
Denied - insufficient privilege
✗ Error 258 ✅PASS

Security Analysis – HDI Roles and Row-Level Security

HDI Roles vs Database Users

A database user is a runtime identity that exists in the HANA system and is created and managed by an administrator. An HDI role (`.hdbrole`) is a design-time artifact that lives in the repository and is created inside an HDI container during deployment. It carries no login. It only describes privileges (schema, object and analytic privileges) and is later granted to users. The benefits are that permissions are written as code, deployed together with the objects they protect, and limited to the container schema, so they cannot leak into other schemas.

Analytic Privilege Effectiveness

`sales_region_privilege` restricts `CV_SALES_ANALYSIS` to rows where `REGION` matches Eeurope.. The view is built with `applyPrivilegeType="SQL_ANALYTIC_PRIVILEGE"`, so the filter is enforced by the engine on every query to the view, not by the application. Two limitations were visible in the design. First, the filter only applies when data is read through the view, and `analyst_role` also has `SELECT` on the whole schema, which allows direct reads of `SALES` without the region filter. Second, the region pattern is hard-coded, so each new region or user group needs a new privilege.

Security Best Practices Applied

- **Least privilege:** one role per duty, and the application role limited to the view.
- **Separation of duties:** developer, analyst and service access are kept apart.
- **Security as code:** roles and privileges live in version control and are deployed with the model.
- **Secrets handling:** credentials are kept in a git-ignored `.env`.
- **Central enforcement:** row filtering is done in the database layer.

Comparison with the Previous Security Module

With plain database user management, privileges are granted with SQL statements in the system. They are hard to review, easy to drift between environments, and tied to a named user. With HDI, the same permissions are in `.hdbrole` files that are versioned in Git, reviewed

like any other code, deployed in one step and reusable across containers and landscapes (dev, test, prod). Redeploying also repairs or removes privileges consistently.

When to use which: use HDI roles for everything that belongs to an application or data model. Use database users for people and systems that need to log in, such as administrators, support staff and technical connections. Users still receive the HDI roles, so the two approaches work together.

Real-World Applications

- **analyst_role:** business users and BI tools that need reporting data limited to their own region.
- **app_service_role:** a technical user for an application or service, with access to the view only.
- **developer_role:** development and test containers. It should not be granted to anyone in production.

Collaboration Insights

Granting access to each other was not possible as was described in the task. It had to be done through an administrator.